# Лабораторная работа №3/2 Классы, конструктор и методы (алгоритмические приемы)

В этой части мы разберем некоторые алгоритмические приемы (рецепты), которые можно использовать для решения задач как из этой лабораторной, так и в реальных проектах. Мы разберем внутреннюю работу динамического списка, словаря и связного списка.

## 1. Список

Как известно, когда мы создаем массив, например, int[] arr = new int[10]; в нем есть обязательный атрибут – фиксированный размер. Фиксированный размер означает, что мы его не можем изменить в процессе работы программы, другими словами, когда мы попытаемся записать элемент за пределами массива, то программа вылетит с ошибкой OutOfBoundsException (выход за границы). В связи с этим нужен механизм автоматического расширения массива, а именно класс обертка над массивами.

Обычно массив расширяют в 1,5 раза (50%). По умолчанию упаковка содержит три переменные: размер, массив с элементами, константу – объем по умолчанию (`DEFAULT_CAPACITY`). Все завязано на переменной размера (`int size`), потому что, если мы будем работать с обычным фиксированным размером массива, мы можем, во-первых, запутаться, во-вторых, столкнуться с ошибками, как OutOfBoundsException, так и логическими, например, элементы сотрутся или перезапишутся. Обычная структура в списках выглядит примерно так:




Далее я перечислил функции методов и их сигнатуры. Три перегруженных конструктора класса для создания объекта, метод добавления элемента (`boolean add(Object item)`), удаление (`boolean remove(Object item)`), удаление всех элементов (`boolean removeAll()`), удаление по определенной позиции (`void removeAt(int index)`), установка элемента в определенную позицию (`Object set(Object item, int index)`), получение элемента по определенной позиции (`Object get(int index)`), проверочный метод является ли список пустым (`boolean isEmpty()`), геттер на размер списка (`int getSize()`), переопределенный метод для вывода содержимого списка (`String toString()`), получение индекса элемента (`int indexOf(Object item)`), центровой метод расширения массива (`private void grow()`).

### 1.1. Конструкторы

Ну, давайте создадим список. Для начала, как мы договорились, нужны три переменные:

```
private static final int DEFAULT_CAPACITY = 10;
private int size;  13 usages
private Object[] items;  17 usages
```

Далее для создания класса списка нам потребуется конструктор, а лучше три. Первый конструктор будет просто инициализировать наш класс и создавать массив с объектами, причем размер у него будет равен 10 по умолчанию.

```
public MyList() {  no usages
    this.items = new Object[DEFAULT_CAPACITY];
}
```

Object – это общий класс для всех объектов, он может быть и числом, и строкой, и массивом, и кем или чем угодно. В других языках также есть общие классы, например Any в Kotlin, в нашем любимом python это object и других языках общего назначения что-то подобное присутствует.

Ладно, далее создадим еще два оставшихся конструктора

```
public MyList(int capacity) {  no usages
    if (capacity < 0) throw new IllegalArgumentException("capacity < 0");
    this.items = new Object[capacity];
}
```

Этот конструктор будет создавать класс с заданным изначальным объемом внутреннего массива. Это может быть полезно, когда нужно контролировать потребление памяти или чтобы избежать повторного расширения списка. Тут можно заменить выброс исключения на обычный System.err.

Наконец последний конструктор.

```
public MyList(Object[] items) {  no usages
    this.items = Arrays.copyOf(items, items.length);
    this.size = items.length;
}
```

В нем мы изначально задаем количество элементов и передаем сами элементы. Arrays.copyOf здесь нужен, чтобы отвязать внутренний массив от массива вызывающего – сохранить инварианты списка, инкапсуляцию и предсказуемую семантику add/remove.

## 1.2. Автоматическое расширение

Создадим метод добавления элемента в конец массива. Логика его следующая:

1) Проверить будет ли добавление элемента за границы внутреннего массива, если да – увеличть в 1,5 раза сам массив;
2) Увеличить size на 1;
3) Добавить элемент в массив.

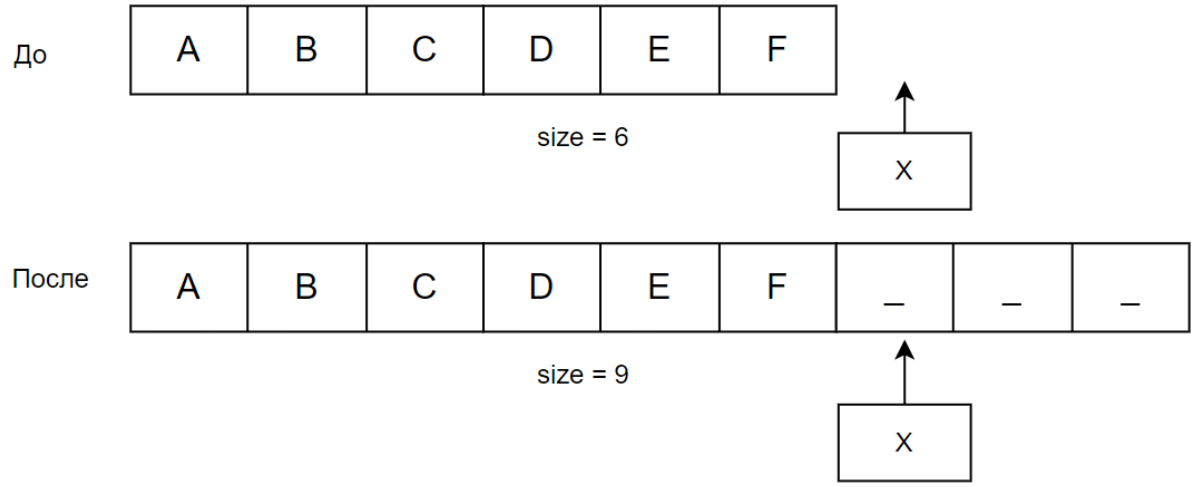
В коде будет выглядеть так:

```java
public boolean add(Object item) {
    if (size >= items.length) {
        grow();
    }
    items[size++] = item;
    return true;
}
```

**Уберите** от экранов Vibe-coders.

```java
private void grow(){  1 usage
    int oldCapacity = items.length;
    int newCapacity = oldCapacity + Math.max(1, oldCapacity >> 1);
    items = Arrays.copyOf(items, newCapacity);
}
```

Тут мы реализуем то самое увеличение примерно в 1,5 раза. Для четных oldCapacity – ровно в 1,5: берём старый размер и прибавляем к нему половину от старого размера. Дальше копируем все элементы в новый массив. Схема расширения и добавления элемента на этом этапе выглядит так:




Зачем Math.max(1, …)? Целочисленное oldCapacity >> 1 при oldCapacity = 0 или 1 даёт ноль. Без Math.max мы бы получили newCapacity = oldCapacity, массив не вырос бы вообще, и add в пустой (или почти пустой) список сломался бы. Так что Math.max гарантирует прибавку хотя бы в один элемент. По сути это краткая форма:

```java
int half = oldCapacity >> 1;
int newCapacity = oldCapacity + (half == 0 ? 1 : half);
```

Про >> 1. Это сдвиг вправо, для неотрицательного int – то же самое, что деление на 2. Исторически сдвиг был быстрее, на современном JIT для неотрицательных значений разницы нет – он и /2, и >>1 сворачивает в одну инструкцию. Разница есть только для отрицательных: -3 >> 1 == -2, а -3 / 2 == -1. Но items.length отрицательным быть не может, так что нам всё равно. Поэтому я написал >> 1 как знак уважения разработчикам: ровно так это написано в ArrayList из OpenJDK.

Также мы можем добавить несколько элементов за раз, скелет метода:

```
public boolean addAll(Object... items) {
    for (Object o : items) {
        add(o);
    }
    return true;
}
```

Троеточие – это **varargs**, тот же массив, но передавать значения мы можем так:

```
MyList m = new MyList();
m.addAll( ...items: 1, 2, 3);
```

Т.е. мы передаем неограниченное количество аргументов. В методе addAll есть несколько ошибок, он написан на «счатливый случай», как минимум нету проверки на null.

## 1.3. Arrays.copyOf

Это тот самый метод, который мы использовали в grow и в конструкторе. По сути этот метод говорит программе: «создай новый массив нужной длины и переложи туда элементы из старого». Не «скопируй в этот же массив», а именно выдели новый. Но есть один НЮАНС. Если внимательно посмотреть на код:

```
MyList list = new MyList(new String[]{"a", "b"});
list.add(42);
```

То видно, что в список передается массив строк, но в него уже добавляется число… это настораживает, потому что при запуске будет следующее:

```
Exception in thread "main" java.lang.ArrayStoreException Create breakpoint : java.lang.Integer
    at org.example.MyList.add(MyList.java:30)
    at org.example.Main.main(Main.java:64)
```

И ошибка любезно отправит нас на строку в методе add

```
public boolean add(Object item) {
    if (size >= items.length) {
        grow();
    }
    items[size++] = item;
    return true;
}
```

Теперь главный вопрос: Почему add и почему ArrayStoreException? Дело в том, что двухаргументный copyOf(items, n) создаёт массив того же runtime-типа, что и items. Если в конструктор пришёл String[], то и после copyOf внутри списка останется String[]. Если бы мы хотели, чтобы наш массив принимал вообще любые типы, то нужно просить программу прямо items = Arrays.copyOf(items, newCapacity, Object[].class); или явно кидать исключение, что не ArrayStoreException, а IllegalArgumentException. Обычно на этом моменте в проде можно потерять день, а может и несколько, если не знать про особеннось Arrays.copyOf.

## 1.4. System.arraycopy

System.arraycopy – это «скопируй из одного куска в кусок другой». Ничего не выделяет. Работает с уже существующими массивами. Именно поэтому сигнатура длинная:

```
@IntrinsicCandidate
public static native void arraycopy( @NotNull @Flow(...) Object src, int srcPos,
                                     @NotNull Object dest, int destPos,
                                     int length);
```

Пять параметров, потому что мы явно указываем: откуда (src), с какого индекса (srcPos), куда (dest), с какого индекса (destPos), сколько элементов.

Нам такой метод нужен, чтобы удалять элементы и эффективно смещать элементы вправо или влево, ну чтобы место выбывшего не пустовало за зря:

```
private void removeAt(int index) {  1 usage
    int moved = size - index - 1;
    if (moved > 0) {
        System.arraycopy(items,  srcPos: index + 1, items, index, moved);
    }
    items[--size] = null;
}
```

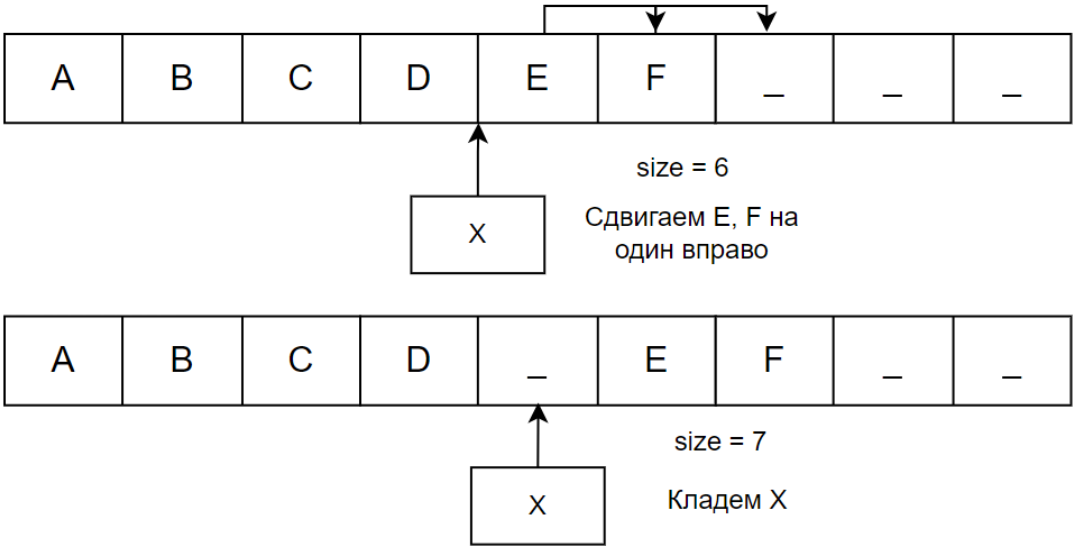
Метод реализован в JVM на C/C++ (отсюда и native в сигнатуре исходника), с оптимизациями под копирование памяти (memmove-подобное поведение). Для больших массивов он в разы быстрее Java-цикла. Для маленьких – JIT всё равно умеет инлайнить цикл, но arraycopy хорош по умолчанию: и быстр, и меньше шансов на ошибку с индексами и перекрытием. Поэтому крайне рекомендую его использовать в задачах подобной этой и вставке элемента в начало, середину списка.

Любимое действие или бездействие разработчиков – убрать items[--size] = null ради «оптимизации», все прекрасно: тесты зеленые, все работает, а после некоторого времени пики на графике профайлера… память «течет». В ArrayList ровно эта же строка (elementData[--size] = null;) с комментарием // clear to let GC do its work.

## 1.5. Вспомогательные методы

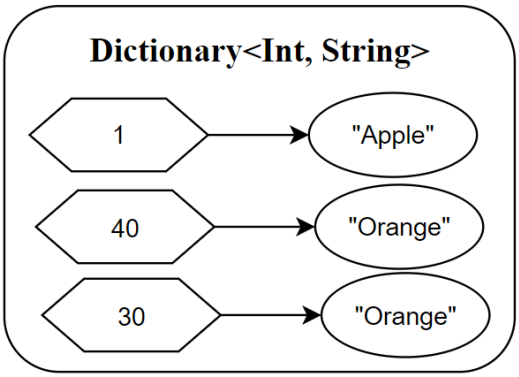
По сути все главные моменты по спискам мы разобрали, осталось сделать вспомогательные методы: возврат размера, проверка на пустоту, замена по индексу, удаление всех элементов. Относительно сложный метод – это вставка элемента на конкретную позицию, но это перегруженный метод add(int idx, Object item), используйте механизм System.arraycopy.

Общая схема будет выглядеть так:

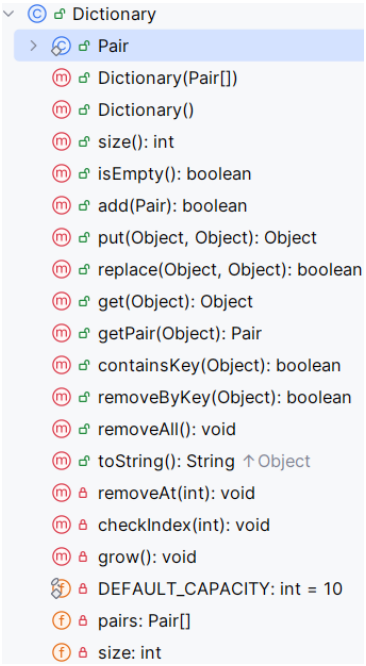



## 2. Словарь

Словарь содержит пары ключ → значение. Причем ключи обычно должны быть уникальны, и пары хранятся не по порядку:




Обычная структура включает, помимо полей и методов, которые очень похожи на те, что мы делали в списке, так же и внутренний класс, обозначающий сами пары:




Методы: size(), isEmpty(), add(Pair pair), get(Object key), removeAll(), toString(), removeAt(int index), checkIndex(int index), grow(), перекочевали из класса MyList практически без изменений, единственное – get, который принимает в качестве аргумента ключ, а не пару.

Это неизменяемый класс Pair (Node):



Во «взрослых» словарях вроде HashMap пары ключ–значение хранятся в хэш-таблице. Это структура данных, устроенная так: из ключа по определённому алгоритму вычисляется число – хэш (hashCode()). Это число сжимается до индекса в массиве корзин (обычно hash & (capacity - 1)). По этому индексу и кладётся пара, отсюда и свойство мапы, что она не хранит элементы по порядку. Но в учебных целях мы сделаем упущение и будем хранить пары как есть, усиленный список.

## 2.1. Вставка элемента по ключу

Вставка элемента происходит в два этапа: первый – проверить существоание элемента по ключу, если есть – обновить элемент в данном ключе иначе добавить новую пару. Если кратко, то это операция upsert – «update or insert», как в оригинальном Map.put в JDK.

```
public Object put(Object key, Object value) {
    Pair existing = getPair(key);
    if (existing != null) {
        Object old = existing.second;
        existing.second = value;
        return old;
    }
    add(new Pair(key, value));
    return null;
}
```

## 2.2. Pair

Вот и ядро нашего словаря – Pair. Внутренний класс, отвечающий за хранение пар. Тут все просто – два поля для хранения ключа и значения, конструктор, геттеры. Сам класс я подсмотрел у андроидовского Pair отсюда и названия полей такие, его также можно переиспользовать в других проектах.

```
public static class Pair {  11 usages
    private Object first;  6 usages
    private Object second;  8 usages

    public Pair(Object first, Object second) {  1 usage
        this.first = first;
        this.second = second;
    }

    public Object getFirst() {  1 usage
        return first;
    }

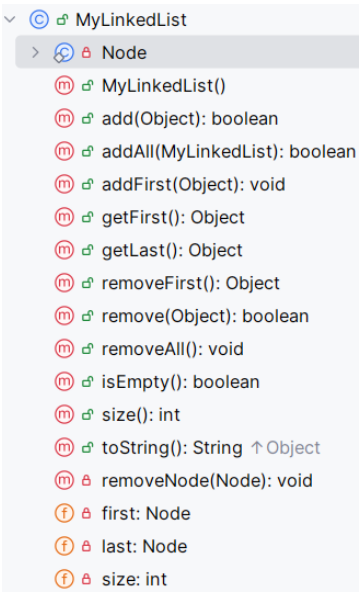
    public Object getSecond() {  1 usage
        return second;
    }

    @Override
    public String toString() {
        return "Pair{first=" + first + ", second=" + second + '}';
    }
}
```

Также можно его модифицировать и сделать не Pair, а Triple, в нем уже хранить по три элемента. Почему класс Pair статический? Ну, вообще все inner-классы статические, так мы явно указываем, что класс внутренний. И не нужно создавать их экземпляр с внешнего класса, т.е. new Pair("a", 1) работает, а не d.new Pair(...); где d – имя словаря.

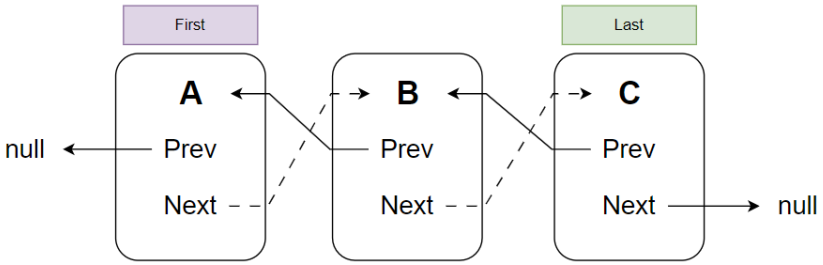
## Двусвязный список

Оказывается, помимо простого списка, есть еще и двусвязный. Зачем его вообще придумали? В обычном списке при удалении или добавлении элементов, особенно это касается центра, мы все элементы сдвигаем относительно удаленного или добавленного элемента, при большом количестве элементов (это когда их миллионы или миллиарды) проблема становится очень заметна – когда у вас зависает программа или начинаются лаги. Эту проблему решили так: давайте будем просто записывать ссылки на следующий и предыдущий элемент, так мы при необходимости отвяжем сслыку и не нужно будет сдвигать их. Удобно? Не знаю, давайте проверим. Ну и по традиции структура:




Названия полей и методов говорят сами за себя, поэтому считаю лишним дублировать их названия и функции. Единственное, это поля first и last – они хранят ссылку на первый и последний элемент самого списка.

Если представить схематично как будет выглядеть список после добаления элементов: A, B, C:




Начнем с inner-класса Node (узел), он будет лаконичным: состоять из трех полей и конструктора:

```
private static class Node {
    private Object data;
    private Node next;
    private Node prev;

    private Node(Object data) {
        this.data = data;
    }
}
```

Node next – ссылка на следующий элемент;
Node prev – ссылка на предыдущий элемент;
Object data – хранимый элемент.

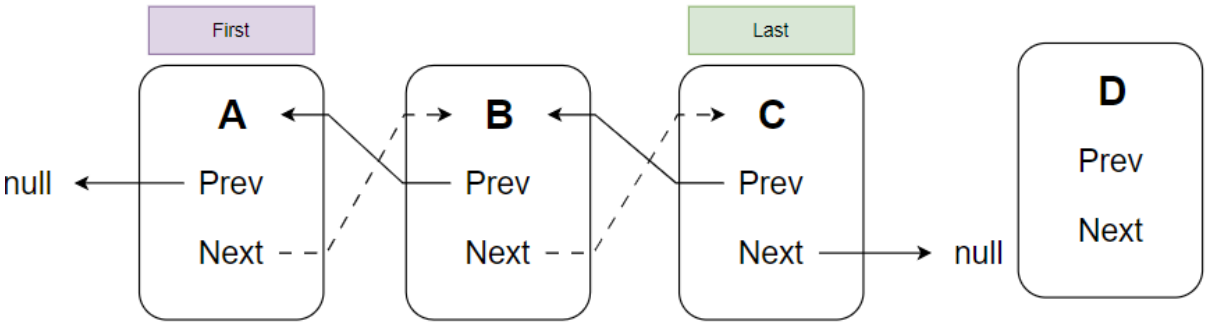
### 3.1. Добавление элементов

Этот алгоритм при добавлении элемента делает следующее: создаёт узел и пришивает его в хвост: если список пуст – узел становится и головой, и хвостом, иначе – старый last начинает указывать на новый узел, новый получает prev = last, а last сдвигается на новый. Затем size++.

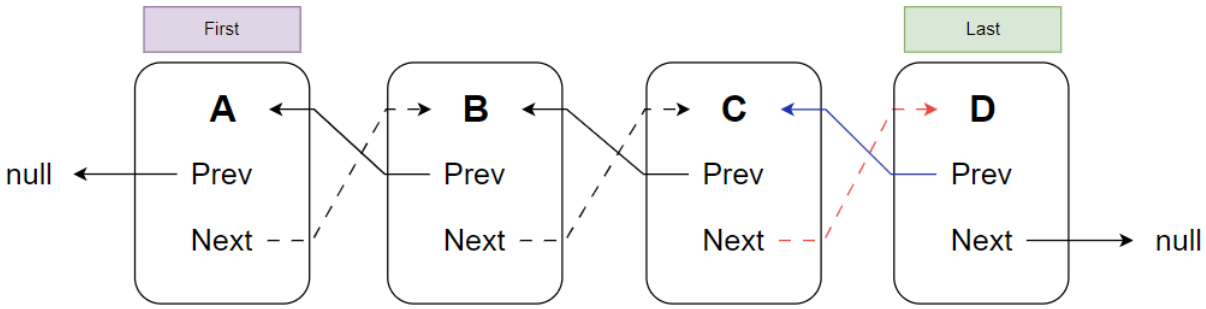
```
Добавляет элемент в конец списка.
public boolean add(Object item) {
    Node newNode = new Node(item);
    if (last == null) {
        // список пуст: новый узел и первый, и последний
        first = last = newNode;
    } else {
        last.next = newNode;
        newNode.prev = last;
        last = newNode;
    }
    size++;
    return true;
}
```

Если нам нужно вставить элемент D, то нужно: создать новый узел, после у C записать в ссылку на следующий (last.next) элемент D, у D записать ссылку на предыдущий (newNode.prev) элемент C, ну и в качестве последнего (last) записать D:

До:




После:




addFirst создаёт узел и пришивает его в голову: если список пуст – узел становится и головой, и хвостом, иначе – новый узел начинает указывать вперёд на старую голову, старая голова получает prev = newNode, а first сдвигается на новый. Затем size++.

```
Добавляет элемент в начало списка.
public void addFirst(Object item) {
    Node newNode = new Node(item);
    if (first == null) {
        first = last = newNode;
    } else {
        newNode.next = first;
        first.prev = newNode;
        first = newNode;
    }
    size++;
}
```

### 3.2. Удаление элементов

removeNode вырезает узел: левого соседа (или first) направляет на node.next, правого (или last) – на node.prev, затем зануляет у узла prev/next, чтобы он не удерживал список, и делает size--. Четыре ветки – это четыре случая: узел в середине, узел - голова, узел – хвост, узел – единственный элемент.

```
private void removeNode(Node node) {  1 usage
    if (node.prev != null) {
        node.prev.next = node.next;
    } else {
        first = node.next; // удаляй первый
    }
    if (node.next != null) {
        node.next.prev = node.prev;
    } else {
        last = node.prev;  // удаляй последний
    }
    node.prev = null;
    node.next = null;
    size--;
}
```

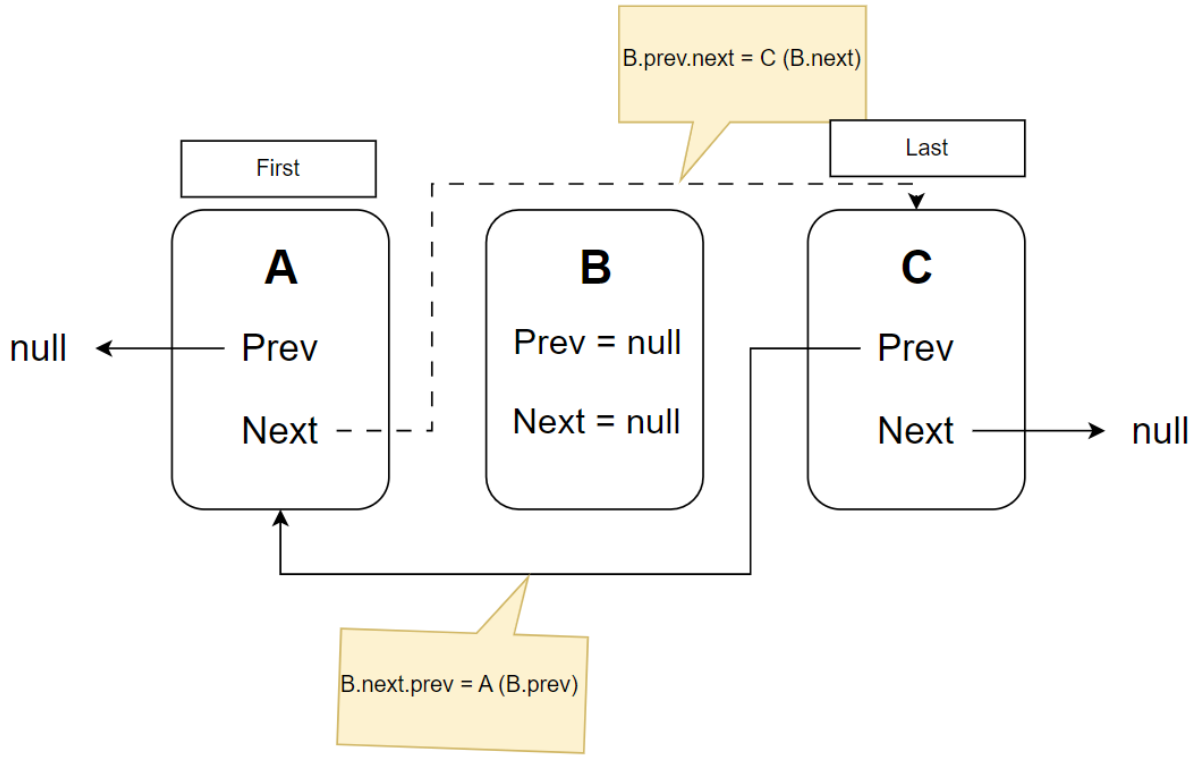
Этот метод удаляет только узел, поэтому нужно сначала найти элемент в узлах через цикл:

```
public boolean remove(Object item) {  1 usage

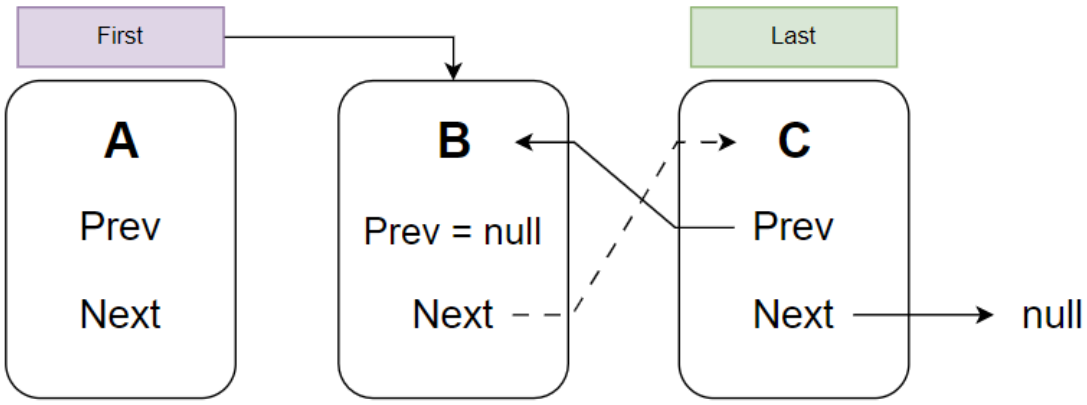
    for (Node node = first; node != null; node = node.next) {
        if (Objects.equals(node.item, item)) {
            removeNode(node);
            return true;
        }
    }

    return false;
}
```

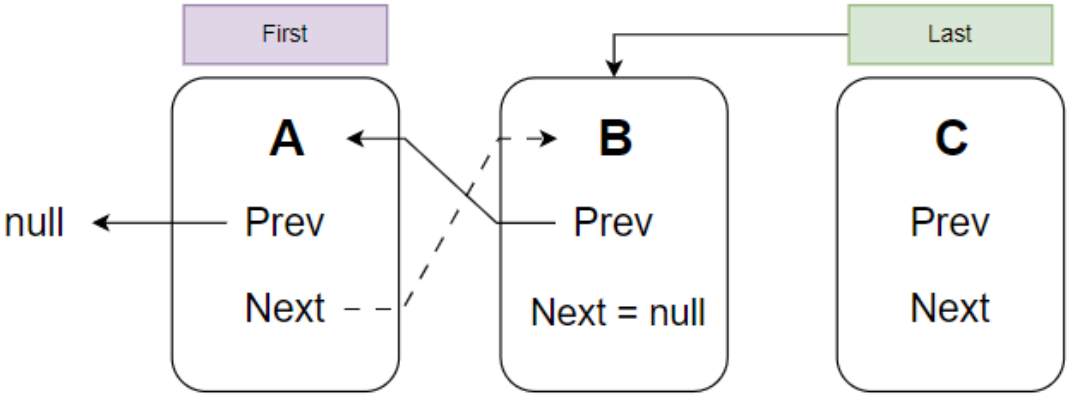
Если представить схематично удаление элемента, это так:




Если мы удаляем элемент из середины (B), то надо перезаписать ссылку на следующий (B.prev – элемент A) элемента A, так чтобы он указываал на C, а ссылку на предыдущий (B.next – элемент C) элемента C, чтобы она указывала на A. Если у нас крайние случаи (начало и конец), то при начале (First) – у первого элемента указать сслыку на следующий:




а при конечном (Last) у последнего записать ссылку на предыдущий:



В принципе это все, что нужно для решения задач из этой лабораторной, задачи C0 и C1 решаются похожим образом, но в них нужно еще дополнительно прогнать в LLM, либо обычным поиском в Интернете, если кто-то еще так ищет, ну или на крайний случай открыть лекции по Алгему или Численным методам. Справка по Методу Гаусса, и еще Гаусс, BFS, DFS, еще BFS. Также прикрепляю источники, по которым я сам когда-то учился создавать Список, Двусвязный список, еще один двусвязный список, ну и бонус ключевые структуры данных (включая хэш-таблицы и динамические списки).

## Задания

Задания выбирать: номер в списке группы mod 4

19 mod 4 = 3

Ограничения:

1) Не использовать дженерики, только класс Object;
2) Не использовать Java Collections Framework;
3) Не использовать лямбда-выражения.
4) Не использовать Arrays.sort();
5) Не использовать сторонние библиотеки.

За реализацию класса из C0 будет присвоено 12 баллов, за C1 – 18 баллов. За доп. задания со звездочкой будет прибавлено 2 балла. Если в прошлой лабораторной сделали C0 или C1 и в этой лабораторной у вас вариант задания 1 или 2, делайте следующий вариант (2 и 3 соответственно).

**0. Создайте класс MyLinkedList** (связный список) с такими полями, как первый элемент в списке, последний элемент в списке, количество элементов.
Реализуйте методы:
1. добавления элемента в список;
2. добавление нового списка;
3. удаление конкретного элемента;
4. удаление всех элементов;
5. добавления элемента в начало;
6. удаления элемента из начала;
7. проверка на пустоту;
8. метод вывода всех элементов в консоль.

Предусмотрите внутренний класс, который будет хранить данные узла списка, ссылку на следующий элемент и на предыдущий. Список может хранить конкретный тип данных, а может и все сразу (Object) (включая ссылочные типы данных).

* Попробуйте решить задачу балансировки скобок или проверку на палиндром из прошлой лабораторной (задачи 4 и 2), используя этот класс.

Пример:

```
// создание списка
MyLinkedList list = new MyLinkedList();
```

```
// добавление элементов по одному
list.add(1);
list.add(2);

// добавление элементов всего списка
list.addAll(new MyLinkedList(1, 2, 3, 4, 5, 6, … ));

// удаление конкретного элемента
list.remove(3);

// определение индекса элемента
list.indexOf(4);

// вывод в консоль
System.out.print(list.toString());

//удаление всех элементов
list.removeAll();
```

**1. Создайте класс MyList (список)** с такими полями, как начальный размер элементов (по умолчанию = 10), массив элементов.
Реализуйте методы:
1. добавления элемента в список;
2. поиск элемента по индексу;
3. удаление всех элементов;
4. удаление конкретного элемента;
5. обновление элемента по индексу;
6. вставка элемента по индексу;
7. получение элемента по индексу;
8. добавление нового списка в существующий;
9. проверка на пустоту;
10. метод вывода всех элементов в консоль.
Предусмотрите приватный метод автоматического расширения внутреннего массива и удалению элемента по индексу. Список может хранить конкретный тип данных, а может и все сразу (Object). Решите задачу палиндрома из предыдущей лабораторной, используя этот класс.

* Попробуйте решить задачу циклического сдвига по аналогии из задачи 7 прошлой лабораторной, используя этот класс.

Пример:
```
// создание списка
MyList list = new MyList();

// добавление элементов по одному
list.add(1);
list.add(2);
```

```
// добавление элементов всего списка
list.addAll(new MyList(1, 2, 3, 4, 5, 6, … ));

// удаление элемента по индексу
list.remove(0);

// вывод в консоль
list.toString();

//удаление всех элементов
list.removeAll();
```

**2. Создайте класс MyUniqueList** (список уникальных элементов) с такими полями, как начальный размер элементов (по умолчанию = 10), массив элементов.
Реализуйте методы:
1. добавления элемента в список
2. добавление нового списка в существующий
3. удаление всех элементов
4. удаление конкретного элемента
5. обновление элемента по индексу
6. получение элемента по индексу
7. проверка на пустоту
8. метод вывода всех элементов в консоль.
Предусмотрите приватный метод автоматического расширения внутреннего массива, а также обеспечьте гарантию, что все элементы в списке будут уникальны. Список может хранить конкретный тип данных, а может и все сразу (Object) (включая ссылочные типы данных).

* Попробуйте найти 2 уникальных элемента в массиве, по аналогии из задачи 8 прошлой лабороторной, используя этот класс.

Пример:
```
// создание списка
MyUniqueList list = new MyUniqueList();

// добавление элементов по одному
list.add(new Person("Vasya"));
list.add(2);

// добавление элементов всего списка
list.addAll(new MyUniqueList(1, 2, 3, 4, 5, 6, … ));

// удаление элемента по индексу
list.remove(0);

// получение элемента по индексу
list.get(4);
```

```
// вывод в консоль
list.toString();

//удаление всех элементов
list.removeAll();
```

**3. Создайте класс Dictionary** (словарь) с такими полями, как массив элементов, состоящих из вложенных классов Pair, размер словаря. Он хранит элементы как пары ключ → значение.
Реализуйте методы:
1. добавления элемента в список;
2. добавление нового словаря в существующий;
3. удаление всех элементов;
4. удаление конкретного элемента;
5. обновление элемента по ключу;
6. получение элемента по ключу;
7. проверка на пустоту;
8. метод вывода всех элементов в консоль.

Что делать с одинаковыми ключами, решить самостоятельно.
Предусмотрите приватный метод автоматического расширения внутреннего массива. Словарь может хранить конкретный тип данных, а может и все сразу (Object) (включая ссылочные типы данных).

* Попробуйте решить задачу подсчета символов у массива по аналогии из задачи 0 прошлой лабороторной.

Пример:
```
// Создание словаря
Dictionary dict = new Dictionary();

// добавление элементов по одному
dict.add(1, “Vasya”);
dict.add(0.5, “Petya”);

// добавление множества элементов
Dictionary second = new Dictionary(
      new Pair(1, “V”),
      new Pair(2, “S”),
      new Pair(3, “Right”)
      …
      …
      …);

dict.put(second)

// удаление элемента по ключу
dict.removeByKey(3);
```

```
// вывод в консоль
list.toString();

//удаление всех элементов
list.removeAll();
```

**С0. Создайте класс MyGraph** (граф) с такими полями, как матрица смежности, массив вершин. Можно заменить матрицу смежности и массив вершин на словарь, чтобы сделать реализацию графа более эффективной и уменьшить память, занимаемую пустыми вершинами, тогда это получится список смежности. Но придется реализовать свой собственный класс Dictionary и MyList.
Реализуйте методы:
1. добавления вершины в граф
2. добавление ребра между вершинами
3. удаление вершины
4. удаление ребра
5. проверка наличия вершины
6. проверка наличия ребра
7. метод вывода всего графа в консоль.

Предусмотрите методы поиска максимального и минимального пути между ребрами, а также существования пути.

Пример:
```
MyGraph graph = new MyGraph();

// добавление вершин
graph.addVertex("A");
graph.addVertex("B");
graph.addVertex("C");
graph.addVertex("D");
graph.addVertex("E");

// добавление рёбер
graph.addEdge("A", "B");
graph.addEdge("A", "C");
graph.addEdge("B", "D");
graph.addEdge("C", "D");
graph.addEdge("D", "E");

// определение существование пути
graph.hasPath("A", "E");

// удаление вершины
graph.removeVertex("C");

// вывод в консоль
graph.toString();
```

```
// определение максимального пути между вершинами
graph.findMaxPath("A", "E");
```

**С1. Написать утилитарный класс Matrix** (матрица) для работы с плотными матрицами. Поля: два двумерных массива. Предусмотреть механизм, при котором пользовватель вводит значения в конструктор, например: “(1, 2\n 3, 4\n 5, 6)”, после программа парсит строку и создает двумерный массив на основе этой введенной строки. Реализоать методы:
1. поэлементное сложение;
2. поэлементное вычитание;
3. поэлементное умножение (предусмотреть A.cols = B.rows);
4. умножение всех элементов на скаляр;
5. транспонирование;
6. поворот на 90° по часовой стрелке;
7. решение системы линейных уравнений для квадратной матрицы методом Гаусса.

```
// создание матриц из строки
Matrix A = new Matrix("(1, 2\n 3, 4)");
Matrix B = new Matrix("(5, 6\n 7, 8)");

System.out.println("A = " + A);
System.out.println("B = " + B);

// поэлементное сложение
System.out.println("\nA + B = " + A.add(B));

// поэлементное вычитание
System.out.println("\nA - B = " + A.subtract(B));

// умножение при A.cols = B.rows
System.out.println("\nA * B = " + A.multiply(B));

// умножение на скаляр
System.out.println("\nA * 2.5 = " + A.scalarMultiply(2.5));

// транспонирование
Matrix C = new Matrix("(1, 2, 3\n 4, 5, 6)");
System.out.println("\nC = " + C);
System.out.println("C^T = " + C.transpose());

// поворот на 90° по часовой стрелке
System.out.println("\nC (rotate 90 CW) = " + C.rotateClockwise());

// решение СЛАУ A*x = b методом Гаусса
// 2x + 3y = 8
// 4x - y = 2
Matrix S = new Matrix("(2,3\n4,-1)");
double[] b = {8,2};
double[] x = S.solve(b);
```

```
System.out.printf("%nSolution: x = %.3f, y = %.3f%n", x[0], x[1]);

// несовместимые размеры — исключение
try {
      Matrix D = new Matrix("(1, 2, 3)");
      A.multiply(D);
} catch (IllegalArgumentException e) {
      System.out.println("\nОшибка: " + e.getMessage());
}
```